

Tristan Niewenhuis: RTOS: Lab 6 Final

Part 1:

The mixer task first sets up the PM to properly use the bit depth and frequency of the given sound data. For the mixer to ISR and ISR to mixer messaging, I used 2 queues that held pointers to 4 static buffers. Starting in the mixer, the mixer will grab a buffer, and copy the effect data onto that buffer, and then send that buffer back to the ISR on the second queue. I was having problems with getting the queue to send the proper pointer to the buffers, as I was accidentally sending the local pointer, then a pointer to memory that didn't exist. But after some trial and error, I was able to get it to work as intended. To actually put the values from the ISR into the device, I would first get the buffer the mixer. I can't block for a buffer like the mixer, so if I don't get one, then I just set the duty to 0. If there is a buffer, then I will start to write to duties one at a time until the FIFO is full. I am keeping track of where I am at in the buffer with a static index. When the index gets to the end of the buffer, I send a used buffer pointer back on the output queue and I try to get the next buffer pointer from the input queue. If that is valid, I just output like before, else I set the duty to 0.

Part 2:

Now to make the mixer wait for an event from the effect tasks, I used an event group. The mixer waits for any change on the event group. Then it uses the returned bit map and checks it against all the designated bits for each event. Then it knows which queues have data to be pulled from. Then because the sound effects are just global arrays, the buffer is just a pointer to the proper section in the sound effect that needs to be mixed in. I add all the data from all sources into a temporary buffer of `int16_t`. Once all the values are added, we will put them into the buffer to the ISR with truncation down to an `int8_t` and add 128 to the value so it can be a `uint8_t` for the duty cycle. The effect task will loop through their entire effect size, trying to put a their pointers into the queue. If the queue is full, it will block until the mixer opens up a spot on the queue. every time it puts a buffer in the queue, it then signals in the event group to wake up the mixer.

Part 3:

To integrate with the ninvadors, I used another group event to signal form the ninvadors tasks to the effect tasks. This was fairly easy to implement, so instead of blocking on a software timer for 2 seconds, it blocks on the bit in the event group. I just had to figure out where to put the effects in the invaders code. I also needed to add some size optimization to the code as it was not longer fitting onto the RAM, but with optimizations it worked.

THEME:

For the theme, I thought it would be easiest to just always play the theme and add it in the mixer without another effect task. To make sure the mixer didn't wait on any effects, instead of waiting for the bit, I just grab the bits without waiting so it can still tell if it needs to add anything from their queues. The hardest part was to get the theme to be in the SRAM appropriately. I was struggling to get the theme to finalize underneath the heap, and instead it was just playing the garbage that was there before. Eventually I was able to figure out the linker script and got it to work.

Successes and Problems:

During my final checks, I was able to get the IDLE task around 75%. The sound task was very small with most of the usage coming from the invaders display task. The theme and all the effects worked perfectly, being able to play multiple all at once with minimal CPU time.

The largest problem I wasn't able to solve was some small jitter that would occasionally happen on the hello_world task. After a fresh startup, it would have 0 jitter for a minute, but sometimes I would get a spike and get jitter in 5 ticks each direction (95 min and 105 max). This was very infrequent only happening once every 60 seconds. I suspected it was the critical sections in the UART put_char task, so I optimized the write_string function to not call put_char over and over to minimize time in the critical section, but that didn't work. Hello world was the highest priority even over timer service, so it must be coming from a ISR or critical section. More tests and trials are needed to fix.

Screenshot of Invaders game with Hello World and stats task.

The top screenshot shows a Konsole window titled "s101101787 : screen — Konsole <2>". It displays a table of statistics:

	Max Time	Min Time	Max Jitter	Count
Hello World	105	95	10	2717
stats	7941	<1%		
IDLE	4543047	85%		
Tmr Svc	13324	<1%		
hello	16312	<1%		
ninvaders	4560	<1%		
effect 0	0	<1%		
effect 5	0	<1%		
effect 6	0	<1%		
effect 8	313	<1%		
effect 7	423	<1%		
effect 2	640	<1%		
effect 3	637	<1%		
effect 4	708	<1%		
effect 1	641	<1%		
mixer	63141	1%		
ninvaders_displ	656511	12%		

The bottom screenshot shows the same Konsole window displaying the Invaders game. The game is in progress, showing a grid of ASCII art representing the game field. The status bar at the bottom indicates: Level: 01 Score: 0000000 Lives: /- \ /- \.